

论文实现思路

课题内容：

1. 基于课程教材构建知识图谱 分析《计算机系统基础》教材内容，初步构建课程知识图谱，为生成式学习助手提供私域知识基础。
2. 基于非结构化图谱构建结构化知识图谱 将从教材、教学资料或文献中提取的非结构化信息进行实体识别，关系抽取和格式化，提取概念、关系和知识点，生成可计算、可查询的结构化知识图谱。
3. 结合大模型和知识图谱实现问答功能 利用大语言模型结合知识图谱，实现学生自然语言问题的理解、查询知识图谱并生成准确回答的功能。
4. 基于私人问答信息更新知识图谱 根据学生的问答交互记录，对知识图谱进行动态更新和扩展，丰富知识点关系，提高问答的准确性与个性化。
5. 测试 设计功能与性能测试，包括问答正确率、响应速度及知识图谱更新有效性验证，确保学习助手稳定可靠。

一.知识图谱的构建

核心流程可以分为四个阶段：**数据获取与清洗** -> **本体（Schema）设计** -> **知识抽取** -> **知识存储**。

1.数据解析和预处理

教材通常是 PDF 格式。需要保留“章节”的层级结构。

- **思路：**

1. 读取 PDF，提取目录（Table of Contents），构建初始的“骨架”树。
2. 按页或按章节提取正文文本。
3. 清洗数据：去除页眉、页脚、页码、乱码。

- **代码逻辑示例：**

```
1 import pdfplumber
2 def extract_text_by_chapter(pdf_path):
3     # 伪代码：实际需要根据PDF目录的坐标或字体大小来判断章节分割点
4     book_content = []
5     with pdfplumber.open(pdf_path) as pdf:
6         for page in pdf.pages:
7             text = page.extract_text()
8             # 简单的清洗
9             clean_text = text.replace("计算机系统基础", "").strip()
10            book_content.append(clean_text)
11    return book_content
```

2. 本体层设计

2.1. 核心节点设计 (Nodes / Labels)

我们需要定义不同类型的节点来承载不同的信息。

A. 文档结构层 (用于 RAG 检索和溯源)

- **Course (课程):** 根节点。例如 {name: "计算机系统基础"}
- **Chapter (章):** 例如 {title: "信息的表示和处理", number: 2}
- **Section (节):** 例如 {title: "整数表示", number: "2.2"}
- **TextChunk (文本块): 这是最重要的节点。**
 - 为了配合大模型 (LLM)，你需要把教材切分成一个个小段落 (比如 500 字一段)。
 - 属性: {content: "...", page_number: 45, embedding: [0.12, -0.5...]}
 - 作用: 当学生提问时，先通过向量检索找到最相关的 TextChunk，再通过图谱找到它关联的知识点。

B. 知识语义层 (用于推理和精准回答)

- **Concept (核心概念):** 最通用的知识点。例如 {name: "补码", alias: ["Two's Complement"]}
- **Hardware (硬件组件):** 计算机系统特有的实体。例如 {name: "ALU", type: "组合逻辑电路"}
- **Instruction (指令):** 汇编指令。例如 {name: "movq", syntax: "movq S, D"}
- **CodeSnippet (代码片段):** 关键的代码案例。例如 {language: "C", content: "int x = -1;"}
- **Principle (原理 / 定理):** 重要的定律。例如 {name: "局部性原理"}

2.2. 核心关系设计 (Relationships)

关系定义了知识的流动方向。

A. 层级与包含关系 (纵向)

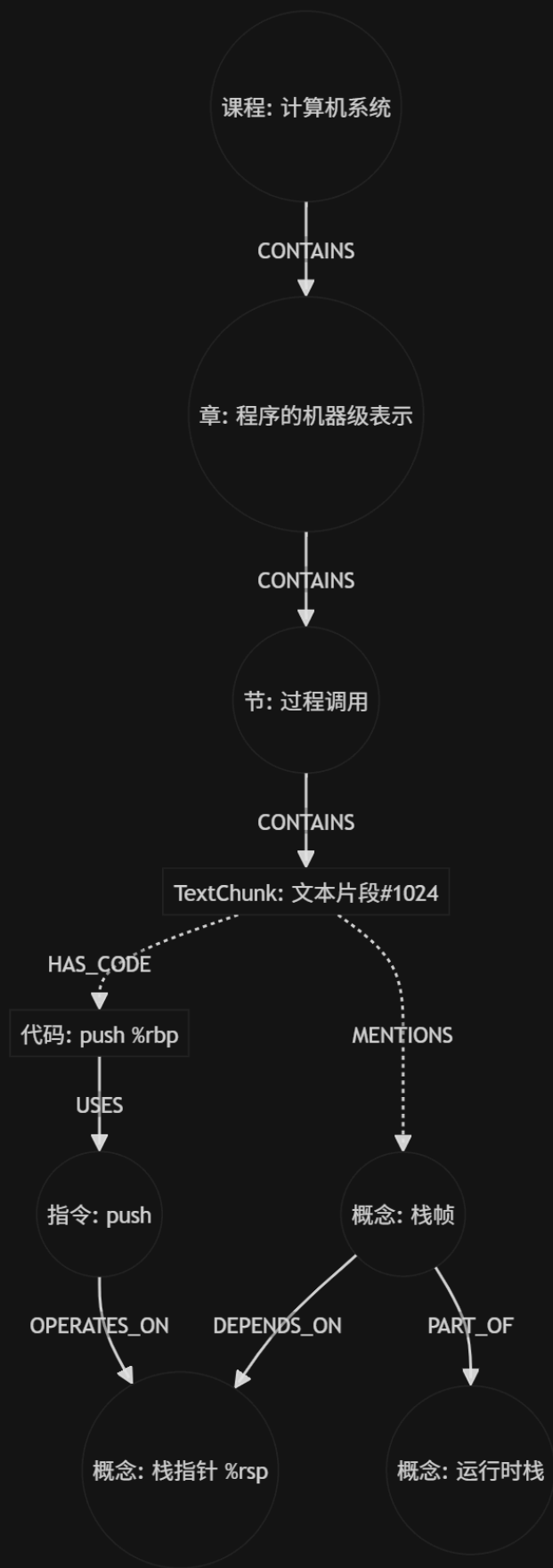
- CONTAINS: 课程 -> 章 -> 节 -> 文本块。
 - (Chapter)-[:CONTAINS]->(Section)
 - (Section)-[:CONTAINS]->(TextChunk)
- MENTIONS: 文本块中提到了哪些概念。
 - (TextChunk)-[:MENTIONS]->(Concept)
- 这是连接“非结构化文本”和“结构化知识”的桥梁。

B. 知识逻辑关系 (横向 - 重点)

对于《计算机系统基础》，这些关系非常有价值：

- DEPENDS_ON (前置依赖): 学习 B 之前必须懂 A。
 - (Concept: 虚拟内存)-[:DEPENDS_ON]->(Concept: 页表)
- IS_A (分类):
 - (Hardware:SRAM)-[:IS_A]->(Concept: 存储器)
- COMPOSED_OF (组成):
 - (Hardware:CPU)-[:COMPOSED_OF]->(Hardware:ALU)
 - (Hardware:CPU)-[:COMPOSED_OF]->(Hardware:PC 寄存器)
- IMPLEMENTED_BY (实现):
 - (Concept: 加法)-[:IMPLEMENTED_BY]->(Hardware: 加法器)
- EXAMPLE_OF (举例):
 - (CodeSnippet)-[:EXAMPLE_OF]->(Concept: 缓冲区溢出)

2.3.推荐的 Schema 可视化图解



2.4.具体属性设计 (Properties) 建议

1. Concept (概念节点)

Python

```
1 graph TD
2     %% 文档层
3     Course((课程: 计算机系统)) -->|CONTAINS| Chapter((章: 程序的机器级表示))
4     Chapter -->|CONTAINS| Section((节: 过程调用))
5     Section -->|CONTAINS| Chunk[TextChunk: 文本片段#1024]
6
7     %% 链接层
8     Chunk -.->|MENTIONS| Concept1((概念: 栈帧))
9     Chunk -.->|HAS_CODE| Code[代码: push %rbp]
10
11    %% 知识层
12    Concept1 -->|DEPENDS_ON| Concept2((概念: 栈指针 %rsp))
13    Concept1 -->|PART_OF| Concept3((概念: 运行时栈))
14    Instruction((指令: push)) -->|OPERATES_ON| Concept2
15    Code -->|USES| Instruction
```

2.TextChunk (文本块节点 - 混合检索核心)

Python

```
1 {
2     "id": "c_1001",
3     "name": "流水线冒险",
4     "description": "流水线中下一条指令无法在指定时钟周期执行的情况。",
5     "importance": 5, // 重要程度 1-5, 用于推荐算法
6     "aliases": ["Pipeline Hazard", "流水线冲突"] // 用于模糊搜索
7 }
```

3.Relation (关系)

```
1 {  
2   "id": "chunk_55",  
3   "text": "数据冒险通常发生在指令之间存在数据依赖关系时...",  
4   "source": "教材_P120",  
5   "embedding": [0.023, -0.11, ...] // 向量数据，用于相似度搜索  
6 }
```

3.利用 LLM 进行知识抽取

- 思路：

我们将采用 **LLM + Schema Constraints (模式约束)** 的方法。相比于传统的 NLP 模型训练（需要大量标注数据），这种方法开发速度快，且对“计算机系统”这种专业领域的理解能力更强。

流程如下：

1. **Schema 定义**：用代码定义好我们要抽取的结构（Pydantic 模型）。
2. **结构化抽取**：让 LLM 严格按照定义输出 JSON 数据。
3. **实体对齐 (Entity Resolution)**：解决“CPU”和“中央处理器”是同一个东西的问题。
4. **写入图数据库**。

1. 定义抽取结构 (使用 Pydantic)

为了防止 LLM 胡乱输出（比如输出一段废话而不是 JSON），我们需要使用 Pydantic 库来强制规定输出格式。这是目前 LangChain 等框架最推荐的做法。

```
1 from typing import List, Optional
2 from pydantic import BaseModel, Field
3
4 # 定义节点结构
5 class KnowledgeNode(BaseModel):
6     id: str = Field(description="实体的唯一标识名称, 尽量简短, 如'虚拟内存'")
7     type: str = Field(description="实体类型, 只能是: Concept, Hardware, Instruction, Code, Principle")
8     desc: Optional[str] = Field(description="实体的简短定义或描述")
9
10 # 定义关系结构
11 class KnowledgeRelationship(BaseModel):
12     source: str = Field(description="源实体的名称")
13     target: str = Field(description="目标实体的名称")
14     type: str = Field(description="关系类型, 只能是: DEPENDS_ON, PART_OF, IS_A, IMPLEMENTED_BY, MENTIONS")
15     reason: Optional[str] = Field(description="建立该关系的理由")
16
17 # 定义最终输出的整体结构
18 class ExtractionResult(BaseModel):
19     nodes: List[KnowledgeNode]
20     relationships: List[KnowledgeRelationship]
```

2. 编写抽取代码 (LangChain + LLM)

```
1 import os
2 from langchain_core.prompts import ChatPromptTemplate
3 from langchain_openai import ChatOpenAI # 或者使用 ChatDeepSeek 等
4 from langchain_core.output_parsers import PydanticOutputParser
5
6 # 1. 初始化大模型 (这里以 OpenAI 协议为例, DeepSeek 也兼容此协议)
7 # 记得在环境变量设置 API_KEY
8 llm = ChatOpenAI(model="gpt-4o", temperature=0) # 温度设为0, 保证输出稳定
9
10 # 2. 设置解析器
11 parser = PydanticOutputParser(pydantic_object=ExtractionResult)
12
13 # 3. 编写 Prompt, 这一步非常关键
14 # 即使使用了 Pydantic, 良好的 Prompt 依然是必须的
15 prompt_template = """
16 你是一位计算机系统基础领域的专家。请分析下面的教材文本片段, 提取其中的核心知识点和它们
    之间的逻辑关系。
17
18 请严格遵守以下规则:
19 1. **节点类型**仅限: Concept(概念), Hardware(硬件), Instruction(指令), Code
    (代码), Principle(原理)。
20 2. **关系类型**仅限: DEPENDS_ON(依赖), PART_OF(组成), IS_A(属于), IMPLEMENTE
    D_BY(实现), MENTIONS(提及)。
21 3. 忽略像"我们要学习..."、"如图所示"这样的非知识性文本。
22 4. 如果出现同义词 (如"Central Processing Unit"和"CPU"), 请统一使用最通用的缩写
    或中文名称作为 id。
23
24 文本内容:
25 {text_chunk}
26
27 {format_instructions}
28 """
29
30 prompt = ChatPromptTemplate.from_template(prompt_template)
31
32 # 4. 构建处理链
33 chain = prompt | llm | parser
34
35 # 5. 执行抽取的函数
36 def extract_structured_data(text_chunk):
37     try:
38         result = chain.invoke({
39             "text_chunk": text_chunk,
```



```

40         "format_instructions": parser.get_format_instructions()
41     })
42     return result
43 except Exception as e:
44     print(f"抽取失败: {e}")
45     return None
46
47 # --- 测试运行 ---
48 sample_text = """
49 流水线 (Pipeline) 是提高 CPU 吞吐率的重要技术。
50 但是, 当后续指令依赖于前一条指令的计算结果时, 会发生数据冒险 (Data Hazard)。
51 解决数据冒险的一种方法是使用转发 (Forwarding) 技术。
52 """
53
54 data = extract_structured_data(sample_text)
55 if data:
56     print("抽取到的节点:", data.nodes)
57     print("抽取到的关系:", data.relationships)

```

3. 实体对齐与清洗 (Entity Resolution)

这是很多初学者容易忽略的一步。LLM 可能会从第一章提取出“Cache”，从第二章提取出“高速缓存”。如果不处理，你的图谱里会有两个没有连接的节点。

简单有效的策略：

在写入数据库之前，维护一个**标准术语表 (Alias Mapping)**。

Python

```

1 # 简单的别名映射字典
2 ALIAS_MAP = {
3     "高速缓存": "Cache",
4     "缓存": "Cache",
5     "CPU": "中央处理器",
6     "Central Processing Unit": "中央处理器",
7     "流水线技术": "流水线"
8 }
9
10 def normalize_entity_name(name):
11     # 去除空格, 转小写 (针对英文) 等
12     clean_name = name.strip()
13     return ALIAS_MAP.get(clean_name, clean_name)

```

进阶做法 (可选): 使用 LLM 专门做一个“归一化”步骤，输入所有提取出的实体名，让 LLM 把它们聚类并选出标准名。

4. 写入 Neo4j (图数据库存储)

最后一步是将清洗后的 Pydantic 对象写入我们在第一步设计好的 Neo4j 结构中。

```

1 from neo4j import GraphDatabase
2
3 class KnowledgeGraphLoader:
4     def __init__(self, uri, user, password):
5         self.driver = GraphDatabase.driver(uri, auth=(user, password))
6
7     def close(self):
8         self.driver.close()
9
10    def save_extraction_result(self, extraction_result, source_chunk_id):
11        """
12        将抽取结果写入 Neo4j, 并将它们链接到原始的 TextChunk
13        """
14        with self.driver.session() as session:
15            # 1. 保存节点
16            for node in extraction_result.nodes:
17                node_id = normalize_entity_name(node.id)
18                session.run(
19                    """
20                    MERGE (n:KnowledgeNode {name: $name})
21                    ON CREATE SET n.type = $type, n.desc = $desc
22                    ON MATCH SET n.last_updated = timestamp()
23                    """,
24                    name=node_id, type=node.type, desc=node.desc
25                )
26
27            # (可选) 建立 TextChunk 到 KnowledgeNode 的 MENTIONS 关系
28            # 这样可以知道这个知识点是哪段文字提取出来的
29            session.run(
30                """
31                MATCH (c:TextChunk {id: $chunk_id})
32                MATCH (k:KnowledgeNode {name: $node_name})
33                MERGE (c)-[:MENTIONS]->(k)
34                """,
35                chunk_id=source_chunk_id, node_name=node_id
36            )
37
38            # 2. 保存关系
39            for rel in extraction_result.relationships:
40                source = normalize_entity_name(rel.source)
41                target = normalize_entity_name(rel.target)
42

```

```

43         # 使用 Cypher 动态构建关系
44         # 注意：在实际生产中，尽量避免动态拼接字符串，这里为了演示简便
45         query = f"""
46         MATCH (a:KnowledgeNode {{name: $source}})
47         MATCH (b:KnowledgeNode {{name: $target}})
48         MERGE (a)-[r:{{rel.type}}]->(b)
49         ON CREATE SET r.reason = $reason
50         """
51         session.run(query, source=source, target=target, reason
    =rel.reason)
52
53 # --- 使用示例 ---
54 # loader = KnowledgeGraphLoader("bolt://localhost:7687", "neo4j", "pass
    word")
55 # loader.save_extraction_result(data, "chunk_001")

```

问题：解决教材内容庞大、上下文易丢失的问题

标准化的“层级上下文锚定流水线”

核心逻辑图解

我们将整个过程分为三个阶段，像流水线一样处理数据：

1. **结构化解析 (Parser)**：把 PDF 拆解成带层级的 JSON 树（而不是纯文本）。
2. **上下文注入 (Context Injector)**：切分文本时，把“它属于哪一章哪一节”作为元数据“锚定”在每一块文本上。
3. **图谱构建 (Graph Builder)**：LLM 抽取 + 实体归一化 + 写入数据库。

最终实现方案：代码级详细步骤

我为你写了一套完整的 Python 伪代码架构，你可以直接照着这个逻辑去填充细节。

1. 数据结构准备

我们定义一个对象来承载“切分后的文本”，重点是多了 metadata 字段。

```
1 # 这是一个增强版的文本块对象
2 class ContextChunk:
3     def __init__(self, text, chapter, section, page_num):
4         self.text = text
5         self.metadata = {
6             "chapter": chapter, # 如 "第2章 信息的表示和处理"
7             "section": section, # 如 "2.1 信息存储"
8             "page": page_num
9         }
10
11 # 生成发送给 LLM 的最终 Prompt 内容
12 def to_prompt_context(self):
13     return f"""
14     [当前上下文]
15     章节: {self.metadata['chapter']}
16     小节: {self.metadata['section']}
17     页码: {self.metadata['page']}
18
19     [正文片段]
20     {self.text}
21     """
```

2. 第一阶段：基于目录的 PDF 解析

不要直接用 `page.extract_text()` 读全文。要利用 PDF 的书签（Outline/Bookmarks）功能。

- **工具**: PyMuPDF (即 fitz)
- **逻辑**: 遍历 PDF 书签，获取每一节的起始页码和结束页码，只提取该范围内的文本。

```
1 import fitz # PyMuPDF
2
3 def parse_pdf_structure(pdf_path):
4     doc = fitz.open(pdf_path)
5     chunks = []
6
7     # 获取目录结构 (level, title, page_num)
8     toc = doc.get_toc()
9
10    # 遍历目录, 提取每一节的内容
11    for i in range(len(toc) - 1):
12        level, title, page = toc[i]
13        next_page = toc[i+1][2] # 下一节的开始页就是这一节的结束页
14
15        # 如果是"小节" (假设 level 2 是小节)
16        if level == 2:
17            # 提取这一节所有页面的文本
18            section_text = ""
19            for p in range(page-1, next_page-1):
20                section_text += doc[p].get_text()
21
22            # --- 关键点: 这里不做切分, 先保存带层级的大块 ---
23            # 我们需要找到它的父级章节 (这里简化处理, 实际需维护一个栈)
24            parent_chapter = "查找上级章节逻辑..."
25
26            # 进入下一步切分
27            sub_chunks = smart_split(section_text, parent_chapter, title,
28                                     page)
29            chunks.extend(sub_chunks)
30    return chunks
```

3. 第二阶段: 智能切分 (Smart Splitting)

使用 LangChain 的切分器, 并将层级信息“注入”到每个块中。

```
1 from langchain.text_splitter import RecursiveCharacterTextSplitter
2
3 text_splitter = RecursiveCharacterTextSplitter(
4     chunk_size=800,      # 800字符一块, 适中
5     chunk_overlap=200,   # 重叠200字, 防止切断逻辑
6     separators=["\n\n", "。", "!", "?", "\n"]
7 )
8
9 def smart_split(long_text, chapter, section, start_page):
10     raw_splits = text_splitter.split_text(long_text)
11
12     enhanced_chunks = []
13     for split in raw_splits:
14         # 封装成我们定义的 ContextChunk
15         chunk = ContextChunk(
16             text=split,
17             chapter=chapter,
18             section=section,
19             page_num=start_page # 简化处理, 实际可估算
20         )
21         enhanced_chunks.append(chunk)
22
23     return enhanced_chunks
```

4. 第三阶段：实体归一化与图谱写入 (Pipeline Execution)

这是解决“实体重复”和“写入 Neo4j”的最终环节。

```

1 from neo4j import GraphDatabase
2 from concurrent.futures import ThreadPoolExecutor
3
4 # 简单的全局实体映射表（实际项目中可以用 向量相似度 或 LLM 辅助维护）
5 ENTITY_MAPPING = {
6     "CPU": "中央处理器",
7     "Central Processing Unit": "中央处理器",
8     "Virtual Memory": "虚拟内存",
9     "VM": "虚拟内存"
10 }
11
12 def normalize(name):
13     return ENTITY_MAPPING.get(name, name)
14
15 def process_single_chunk(chunk, neo4j_driver):
16     # 1. 构造 Prompt（包含 metadata）
17     prompt_text = chunk.to_prompt_context()
18
19     # 2. 调用 LLM（复用之前的 extract_structured_data 函数）
20     # 注意：这里传入的是带上下文的 prompt_text
21     data = extract_structured_data(prompt_text)
22
23     if not data: return
24
25     # 3. 写入 Neo4j
26     with neo4j_driver.session() as session:
27         # 3.1 建立"文档层"节点（TextChunk）
28         chunk_id = f"{chunk.metadata['section']}_{hash(chunk.text)}"
29         session.run("""
30             MERGE (c:TextChunk {id: $id})
31             SET c.content = $text,
32                 c.chapter = $chapter,
33                 c.section = $section
34             """, id=chunk_id, text=chunk.text, **chunk.metadata)
35
36         # 3.2 建立"知识层"节点与关系
37         for node in data.nodes:
38             std_name = normalize(node.id) # 归一化
39
40             # 创建知识节点
41             session.run("""
42                 MERGE (k:KnowledgeNode {name: $name})
43                 ON CREATE SET k.type = $type, k.desc = $desc

```



```

44         """", name=std_name, type=node.type, desc=node.desc)
45
46         # 建立 文本->知识 的链接 (MENTIONS)
47         session.run("""
48             MATCH (c:TextChunk {id: $cid}), (k:KnowledgeNode {name: $kname})
49             MERGE (c)-[:MENTIONS]->(k)
50             """", cid=chunk_id, kname=std_name)
51
52     # 3.3 建立知识点之间的关系
53     for rel in data.relationships:
54         src = normalize(rel.source)
55         tgt = normalize(rel.target)
56         session.run(f"""
57             MATCH (a:KnowledgeNode {{name: $src}}), (b:KnowledgeNode {{name: $tgt}})
58             MERGE (a)-[:{rel.type}]->(b)
59             """", src=src, tgt=tgt)
60
61 # 主程序入口
62 def main():
63     # 1. 解析目录结构, 获取切分好的块列表
64     all_chunks = parse_pdf_structure("cs_book.pdf")
65
66     # 2. 初始化数据库
67     driver = GraphDatabase.driver("bolt://localhost:7687", auth=("neo4j", "password"))
68
69     # 3. 并行处理 (加快速度)
70     # 建议 DeepSeek/GPT-3.5 并发处理, GPT-4 需要控制速率
71     with ThreadPoolExecutor(max_workers=5) as executor:
72         for chunk in all_chunks:
73             executor.submit(process_single_chunk, chunk, driver)
74
75     print("构建完成!")

```

1. 先不要跑全书。
2. 手动复制《计算机系统基础》某一节（例如“2.1 信息存储”）的文本。
3. 用我上面的 ContextChunk 类封装一下。
4. 跑通 process_single_chunk 函数，看看 Neo4j 里生成的图是不是符合预期。

如果测试通过，再写 PDF 解析脚本去跑全书。你需要我先帮你写一段测试脚本吗？

4.知识融合与存储

LLM 提取的结果可能会有重复（例如 Chunk A 提取了“缓存”，Chunk B 也提取了“Cache”，需要做对齐，或者简单的去重）。

- **思路：**

1. 连接 Neo4j 数据库。
2. 遍历抽取到的 JSON 列表。
3. 使用 MERGE 语句（Neo4j 的语法，意为“有则匹配，无则创建”）来避免重复节点。

- **代码逻辑示例 (py2neo)：**

Python

```
1  from py2neo import Graph, Node, Relationship
2
3  # 连接数据库
4  graph = Graph("bolt://localhost:7687", auth=("neo4j", "password"))
5
6  def save_to_neo4j(triples):
7      tx = graph.begin()
8      for item in triples:
9          # 创建或匹配头节点
10         head_node = Node(item['type'], name=item['head'])
11         tx.merge(head_node, item['type'], "name")
12
13         # 创建或匹配尾节点
14         tail_node = Node(item['tail_type'], name=item['tail'])
15         tx.merge(tail_node, item['tail_type'], "name")
16
17         # 创建关系
18         rel = Relationship(head_node, item['relation'], tail_node)
19         tx.merge(rel)
20     tx.commit()
```

5.最终步骤

一、 核心架构设计：双层图谱模型 (Dual-Layer Graph Schema)

我们在 Neo4j 中构建 “文档” 与 “知识” 并行的双层结构。这种结构既能支持精确的**原文检索** (RAG)，又能支持深度的**知识推理**。

1. 图谱 Schema 定义

- 文档层 (Document Layer):** 用于物理定位，回答 “知识出自哪里”。
- 节点: Course (课程) → Chapter (章) → Section (节) → TextChunk (文本切片)
- 知识层 (Knowledge Layer):** 用于逻辑关联，回答 “知识是什么关系”。
- 节点: Concept (概念), Hardware (硬件组件), Instruction (指令), Code (代码片段), Principle (原理)
- 连接边:**
- MENTIONS: TextChunk → Concept (连接两层的桥梁)
- 逻辑关系: DEPENDS_ON (依赖), PART_OF (组成), IS_A (属于), IMPLEMENTED_BY (实现)

二、实现流程：层级上下文锚定流水线 (Hierarchical Context Anchoring Pipeline)

请按照以下四个严格的步骤编写代码。

步骤 1：基于目录结构的 PDF 解析 (Hierarchical Parsing)

目标: 利用 PDF 的书签 (Outline) 提取文本，而不是直接转纯文本，保留 “章 - 节” 层级。

工具: PyMuPDF (import fitz)

- 操作逻辑:**
- 1. 读取 PDF 的 TOC (Table of Contents)。
- 2. 获取每一节 (Section) 的起始页和结束页。
- 3. 提取该范围内的文本，并封装成对象，标记其属于哪个章节。

步骤 2：带元数据的上下文切分 (Context-Aware Chunking)

目标: 解决 “切分后丢失上下文” 的问题。

工具: LangChain

- 操作逻辑:**
- 1. 使用滑动窗口 (如 Size=800, Overlap=200) 切分文本。
- 2. **关键动作:** 在生成 Prompt 前，将步骤 1 获取的 “章标题” 和 “节标题” 强行拼接到切分块的头部。
- 3. **效果:** LLM 处理一段关于 “它” 的文字时，能通过标题知道 “它” 指的是 “缓存” 还是 “页表”。

步骤 3：结构化知识抽取 (Structured Extraction)

目标: 强制 LLM 输出标准 JSON，避免格式错误。

工具: LangChain + Pydantic + OpenAI/DeepSeek API

- 操作逻辑:**
- 1. 定义 Pydantic Class (KnowledgeNode, KnowledgeRelation)。

2. 设计 Prompt，要求 LLM 提取实体和关系，并进行**初步归一化**（如将 “Central Processing Unit” 统一标为 “CPU”）。

步骤 4：图数据库写入与融合 (Ingestion & Fusion)

目标： 存入 Neo4j 并去重。

工具： neo4j (Python Driver)

- **操作逻辑：**
 1. 使用 Cypher 的 MERGE 语句，确保同一个概念（如 “虚拟内存”）在数据库中只生成一个节点。
 2. 同时创建 TextChunk 节点，并建立 (TextChunk)-[:MENTIONS]->(Concept) 关系。

```

1 import fitz # PyMuPDF
2 from langchain.text_splitter import RecursiveCharacterTextSplitter
3 from langchain_openai import ChatOpenAI
4 from neo4j import GraphDatabase
5 from pydantic import BaseModel, Field
6 from typing import List
7
8 # --- 1. 定义数据结构 (Schema) ---
9 class KnowledgeNode(BaseModel):
10     id: str = Field(description="标准化的实体名称, 如 'CPU'")
11     type: str = Field(description="类型: Concept, Hardware, Instruction, Code")
12     desc: str = Field(description="简短定义")
13
14 class KnowledgeRelation(BaseModel):
15     source: str
16     target: str
17     type: str = Field(description="DEPENDS_ON, PART_OF, IS_A, IMPLEMENTED_BY")
18
19 class ExtractionResult(BaseModel):
20     nodes: List[KnowledgeNode]
21     relations: List[KnowledgeRelation]
22
23 # --- 2. 核心处理类 ---
24 class CSAPPGraphBuilder:
25     def __init__(self, pdf_path, neo4j_uri, neo4j_auth, openai_key):
26         self.pdf_path = pdf_path
27         self.driver = GraphDatabase.driver(neo4j_uri, auth=neo4j_auth)
28         self.llm = ChatOpenAI(api_key=openai_key, model="gpt-4o", temperature=0)
29         # 强结构化输出模式
30         self.structured_llm = self.llm.with_structured_output(ExtractionResult)
31         self.splitter = RecursiveCharacterTextSplitter(chunk_size=800, chunk_overlap=200)
32
33     def parse_and_process(self):
34         """主流程: 解析 -> 切分 -> 抽取 -> 存储"""
35         doc = fitz.open(self.pdf_path)
36         toc = doc.get_toc() # 获取目录 [[lvl, title, page], ...]
37
38         # 遍历目录的每一节

```

```

39         for i in range(len(toc) - 1):
40             level, title, page_start = toc[i]
41             page_end = toc[i+1][2]
42
43             if level == 2: # 假设我们要处理的是"节" (Section)
44                 # 1. 获取章节层级名称 (简单逻辑: 向上找level=1的父级)
45                 chapter_title = "第X章..." # 需实现简单的查找父级逻辑
46
47                 # 2. 提取纯文本
48                 text_content = ""
49                 for p in range(page_start-1, page_end-1):
50                     text_content += doc[p].get_text()
51
52                 # 3. 处理该节内容
53                 self._process_section(text_content, chapter_title, title)
54
55     def _process_section(self, text, chapter, section):
56         """处理单个小节: 切分并抽取"""
57         chunks = self.splitter.split_text(text)
58
59         for chunk_text in chunks:
60             # --- 关键点: 上下文注入 ---
61             # 这里的 prompt_context 包含层级信息, 发给 LLM
62             prompt_context = f"章节: {chapter}\n小节: {section}\n内容: {chunk_text}"
63
64             # 4. 调用 LLM 抽取
65             try:
66                 result = self.structured_llm.invoke(f"从以下计算机系统教材片段中提取知识图谱: \n{prompt_context}")
67
68                 # 5. 存入 Neo4j
69                 self._save_to_neo4j(result, chunk_text, chapter, section)
70
71             except Exception as e:
72                 print(f"抽取错误: {e}")
73
74     def _save_to_neo4j(self, data, original_text, chapter, section):
75         """使用 MERGE 避免重复"""
76         with self.driver.session() as session:
77             # 1. 存储文档片段 (TextChunk)
78             chunk_id = f"{section}_{hash(original_text)}"
79             session.run("""
80                 MERGE (c:TextChunk {id: $id})
81                 SET c.content = $text, c.chapter = $chap, c.section =

```

```

$sec
81         """", id=chunk_id, text=original_text, chap=chapter, sec=sec
      tion)
82
83         # 2. 存储实体 (KnowledgeNode)
84         for node in data.nodes:
85             session.run("""
86                 MERGE (n:KnowledgeNode {name: $name})
87                 ON CREATE SET n.type = $type, n.desc = $desc
88                 """, name=node.id, type=node.type, desc=node.desc)
89
90         # 建立 文档->实体 的关联
91         session.run("""
92             MATCH (c:TextChunk {id: $cid}), (n:KnowledgeNode {n
ame: $name})
93             MERGE (c)-[:MENTIONS]->(n)
94             """, cid=chunk_id, name=node.id)
95
96         # 3. 存储关系
97         for rel in data.relations:
98             session.run("""
99                 MATCH (a:KnowledgeNode {name: $src}), (b:KnowledgeN
ode {name: $tgt})
100                 MERGE (a)-[:DEPENDS_ON {type: $type}]->(b)
101                 """, src=rel.source, tgt=rel.target, type=rel.type)
102
103 # 使用示例
104 # builder = CSAPPGraphBuilder("book.pdf", "bolt://localhost:7687", ("ne
o4j", "password"), "sk-xxx")
105 # builder.parse_and_process()

```

二.大模型问答的实现

第三步是整个系统的“大脑”部分。这一步的核心目标是：当学生提问时，不仅要像搜索引擎一样找到原文，还要像老师一样利用知识结构进行推理。

在学术界和工业界，目前最先进的解决方案是 **GraphRAG**（基于知识图谱的检索增强生成）。

我们可以将其拆解为“**双路召回 + 混合推理**”的技术路线。

1. 核心逻辑设计：GraphRAG workflow

当学生问：“为什么流水线会出现数据冒险？”时，系统按以下步骤工作：

1. 意图理解与实体链接 (Query Understanding)：

- 识别关键词：“流水线 (Pipeline)”、“数据冒险 (Data Hazard)”。
 - 映射到 Neo4j 中的 Concept 节点。
2. **双路召回 (Hybrid Retrieval):**
- **A 路：非结构化检索 (Vector Search):** 在 TextChunk 节点中搜索与问题语义相似的教材原文片段。
 - *作用：* 提供详实的定义、描述和上下文。
 - **B 路：结构化检索 (Graph Traversal):** 从识别出的实体出发，查找它的邻居节点（比如 DEPENDS_ON, CAUSES 关系）。
 - *作用：* 发现“数据依赖”导致“冒险”，以及“转发”是解决办法。这些逻辑可能在原文中相隔很远，但图谱把它们连在一起了。
3. **上下文重组 (Context Assembly):**
- 将“原文片段”和“图谱关系”拼接成一段 Prompt。
4. **生成回答 (Generation):**
- 大模型基于拼接好的丰富信息生成最终答案。

2. 具体代码实现 (基于 Python + LangChain)

我们需要封装一个 GraphRAGEngine 类。

第一步：准备工作 (向量化)

在第二步中，我们建立了 TextChunk 节点。现在我们需要确保这些节点有**向量索引**。

(如果在第二步写入时没存向量，这里需要补充一个脚本，把 content 字段转成 embedding 存入 Neo4j 的 Vector Index)。

Python

```
1 from langchain_openai import OpenAIEmbeddings
2 from langchain_community.vectorstores import Neo4jVector
3
4 # 初始化向量检索工具 (自动连接 Neo4j)
5 vector_store = Neo4jVector.from_existing_graph(
6     embedding=OpenAIEmbeddings(),
7     url="bolt://localhost:7687",
8     username="neo4j",
9     password="password",
10    index_name="text_chunk_index",
11    node_label="TextChunk",
12    text_node_property="content",
13    embedding_node_property="embedding",
14 )
```

第二步：实现问答引擎

这是核心代码，包含了实体提取、图检索、混合 Prompt 构建。


```

1 from langchain_openai import ChatOpenAI
2 from neo4j import GraphDatabase
3
4 class CSAPP_QA_System:
5     def __init__(self):
6         self.driver = GraphDatabase.driver("bolt://localhost:7687", aut
h=("neo4j", "password"))
7         self.llm = ChatOpenAI(model="gpt-4o", temperature=0.2) # 低温度保
证严谨
8         self.vector_store = vector_store # 上面初始化的向量库
9
10    def _get_graph_context(self, entities):
11        """
12        结构化检索：查找实体周围的一跳或两跳邻居关系
13        """
14        context_lines = []
15        with self.driver.session() as session:
16            for entity in entities:
17                # 查询 Cypher：查找该实体相关的关系（出边和入边）
18                query = """
19                MATCH (n:KnowledgeNode {name: $name})-[r]-(m:KnowledgeN
ode)
20                RETURN type(r) as rel_type, m.name as target_name, m.de
sc as target_desc
21                LIMIT 5
22                """
23                result = session.run(query, name=entity)
24
25                for record in result:
26                    # 格式化为自然语言： "流水线 DEPENDS_ON 指令集架构"
27                    line = f"{entity} --[{record['rel_type']}]--> {reco
rd['target_name']} ({record['target_desc']})"
28                    context_lines.append(line)
29
30        return "\n".join(context_lines)
31
32    def _extract_entities(self, question):
33        """
34        利用 LLM 简单提取问题中的关键词，用于去图谱里查
35        """
36        prompt = f"从问题中提取计算机系统领域的专业术语实体，只返回列表，不要其他
内容。问题：{question}"
37        response = self.llm.invoke(prompt)

```

```

38         # 简单处理返回，假设返回是 "流水线, 数据冒险"
39         return [x.strip() for x in response.content.split(",")]
40
41     def answer_question(self, question):
42         # 1. [A路] 向量检索：找教材原文
43         # k=3 表示找最相关的3段原文
44         docs = self.vector_store.similarity_search(question, k=3)
45         text_context = "\n\n".join([d.page_content for d in docs])
46
47         # 2. [B路] 图谱检索：找逻辑关系
48         entities = self._extract_entities(question)
49         graph_context = self._get_graph_context(entities)
50
51         # 3. 构建最终 Prompt（混合上下文）
52         final_prompt = f"""
53         你是一个《计算机系统基础》的助教。请根据以下提供的【教材原文】和【知识图谱逻辑】来回答学生的问题。
54
55         问题: {question}
56
57         === 知识图谱逻辑（结构化关系） ===
58         {graph_context}
59
60         === 教材原文片段（非结构化细节） ===
61         {text_context}
62
63         回答要求：
64         1. 结合两者信息，逻辑清晰。
65         2. 如果图谱中指出了依赖关系或因果关系，请在回答中强调。
66         3. 引用原文中的具体定义。
67         """
68
69         # 4. 生成回答
70         response = self.llm.invoke(final_prompt)
71         return response.content
72
73 # --- 使用示例 ---
74 # bot = CSAPP_QA_System()
75 # answer = bot.answer_question("什么是数据冒险？如何解决？")
76 # print(answer)

```

3. 这一步在论文中的亮点分析

在撰写论文第3部分时，你应该重点突出以下“**创新点**”（相比于普通的 ChatGPT）：

1. 可解释性与准确性 (Grounding):

- 你的系统不是瞎编的，而是基于 {text_context}（教材原文）生成的。如果是幻觉，可以通过检查原文片段来定位。
 - 论文话术：“通过引入 RAG 技术，有效抑制了大模型的幻觉问题，确保回答忠实于《计算机系统基础》教材。”
2. **逻辑推理能力 (Reasoning):**
- 普通的向量搜索只能找到包含“数据冒险”关键词的段落，可能找不到“转发”这个词（因为文中可能相隔很远）。
 - 但是你的图谱里有 (数据冒险)-[:SOLVED_BY]->(转发)。
 - 论文话术：“利用知识图谱的路径跳跃能力，系统能够回答跨段落、跨章节的复杂逻辑问题，这是传统关键词搜索无法做到的。”
3. **个性化深度:**
- 你可以根据 KnowledgeNode 的层级，决定回答的深浅。

三.知识图谱的更新

第四步 “**基于私人问答信息更新知识图谱**” 是你的系统从“静态字典”进化为“动态大脑”的关键。这体现了系统的**自适应性和个性化**。

这一步的实现核心在于建立一个“**交互反馈闭环**” (Interaction-Feedback Loop)。

1. 设计思路：从交互中挖掘什么？

学生的每一次提问和反馈，都蕴含着两类有价值的信息：

- 高频共现 (Co-occurrence):** 如果很多学生在问“流水线”的时候，紧接着问“分支预测”，说明这两个知识点在学生的认知路径上是强相关的，即使教材里没明写，图谱里也应该连起来。
- 未命中补充 (Missed Knowledge):** 如果学生问了一个概念，图谱里没有（检索结果为空），但大模型根据通用知识回答了。这时候应该把这个新概念“捕捉”并写入图谱。
- 个性化偏好 (User Profile):** 记录某个学生已经掌握了哪些知识点，构建“学生画像子图”。

2. 技术架构：异步更新机制

为了不影响问答速度，图谱更新不能实时同步进行，而应该是一个**后台异步任务**。

架构图解:

学生提问 -> 生成回答 -> [存储日志到数据库] -> (后台定时任务) -> 分析日志 -> 更新 Neo4j

3. 具体代码实现步骤

第一阶段：记录交互日志 (Logging)

我们需要在 Neo4j 中引入新的节点类型 User 和 Interaction。

Python

```
1 def log_interaction(user_id, question, answer, involved_entities):
2     """
3     在问答结束后调用。
4     在图谱中记录: (User)-[:ASKED]->(Question)-[:ABOUT]->(Concept)
5     """
6     with driver.session() as session:
7         session.run("""
8             MERGE (u:User {id: $uid})
9             CREATE (q:Question {text: $text, timestamp: datetime()})
10            CREATE (u)-[:ASKED]->(q)
11
12            // 建立问题与相关概念的联系
13            WITH q
14            UNWIND $entities as entity_name
15            MATCH (c:KnowledgeNode {name: entity_name})
16            MERGE (q)-[:ABOUT]->(c)
17            """, uid=user_id, text=question, entities=involved_entities)
```

第二阶段：挖掘隐含关系 (Mining Implicit Relations)

编写一个分析脚本，定期（如每天凌晨）运行。分析逻辑：**如果两个概念经常在同一个问题中出现，或者被连续提问，它们之间可能存在“学习关联”。**

```
1 def update_graph_from_logs():
2     """
3     挖掘高频共现关系，建立 [RELATED_T0] 边
4     """
5     with driver.session() as session:
6         # 查找同时出现在同一个问题中的概念对，且次数超过阈值(例如3次)
7         query = """
8         MATCH (q:Question)-[:ABOUT]->(c1:KnowledgeNode)
9         MATCH (q)-[:ABOUT]->(c2:KnowledgeNode)
10        WHERE c1.name < c2.name // 避免重复 (A,B) 和 (B,A)
11        WITH c1, c2, count(q) as freq
12        WHERE freq > 3
13
14        // 如果它们之间原本没有关系，则建立一个"弱关联"
15        MERGE (c1)-[r:RELATED_T0]-(c2)
16        ON CREATE SET r.weight = 1, r.source = 'user_feedback'
17        ON MATCH SET r.weight = r.weight + 1
18        """
19        session.run(query)
20        print("图谱关联更新完成。")
```

第三阶段：捕获新知识 (Knowledge Discovery)

如果大模型在回答中生成了图谱里没有的概念，我们可以尝试自动捕获。

- **实现逻辑：**

1. 利用 LLM 分析生成的 Answer。
2. 提取其中的关键实体。
3. 检查这些实体是否已在 Neo4j 中。
4. 如果不在，创建一个标记为 source: "generated" 的新节点，待人工审核或自动保留。

```
1 def discover_new_concepts(question, answer):
2     # 1. 让 LLM 从由于回答中提取概念
3     new_concepts = llm_extract_concepts(answer)
4
5     with driver.session() as session:
6         for concept in new_concepts:
7             # 检查是否存在
8             result = session.run("MATCH (n:KnowledgeNode {name: $name}) RETURN n", name=concept)
9             if result.peek() is None:
10                 # 创建新节点 (打上待审核标记)
11                 session.run("""
12                     CREATE (n:KnowledgeNode {name: $name, type: 'Concept', status: 'pending_review'})
13                     SET n.discovered_from = $q
14                     """, name=concept, q=question)
```

第四阶段：个性化推荐 (Personalization)

这是论文加分项。通过分析用户的 ASKED 历史，可以推断他的薄弱点。

- **应用场景：**当学生再次提问时，Prompt 可以加入：“注意：该学生之前多次询问过关于‘虚拟内存’的问题，请在回答中重点复习相关基础。”

```

1 def get_user_profile_context(user_id):
2     """
3     获取用户的"关注点热力图"
4     """
5     with driver.session() as session:
6         result = session.run("""
7             MATCH (u:User {id: $uid})-[:ASKED]->(q)-[:ABOUT]->(c:KnowledgeNode)
8             RETURN c.name, count(q) as focus_count
9             ORDER BY focus_count DESC LIMIT 3
10        """, uid=user_id)
11
12        topics = [record['c.name'] for record in result]
13        if topics:
14            return f"用户高频关注领域: {'、'.join(topics)}。请适当关联这些旧知识。"
15        return ""

```

4. 论文中的表述策略

在撰写这一章时，用以下术语来提升学术性：

1. “人在回路的知识进化” (Human-in-the-loop Knowledge Evolution)：强调图谱不是死的，而是随着人的使用而生长的。
2. “群体智慧挖掘” (Collective Intelligence Mining)：通过分析全班学生的提问，发现教材中容易让人困惑的“隐性关联”。
3. “自适应学习路径” (Adaptive Learning Path)：基于个性化历史记录，动态调整回答策略。

总结方案

第四步的实现并不需要极其复杂的算法，关键在于数据流的设计：

1. 写：把问答过程结构化地存回图谱 (User -> Question -> Concept)。
2. 算：后台脚本计算共现频率 (Co-occurrence)。
3. 读：下次回答时，读取用户画像，实现个性化。

这样，你的系统就形成了一个完美的闭环。至此，你的课题 1-4 步的技术路线已经完全打通。